

CSCE 641 COMPUTER GRAPHICS · MODULE 04 · LECTURE NOTES

Rasterization: From Triangles to Fragments

Deciding which pixels a triangle covers, and what they inherit.

MODULE OUTCOMES

- Implement triangle coverage using edge functions.
- Apply a fill rule that makes shared edges neither doubled nor gapped.
- Derive barycentric coordinates and use them to interpolate attributes.
- Explain why screen-space linear interpolation is wrong, and correct it.
- Rasterize incrementally rather than re-evaluating per pixel.

1 The coverage question

Rasterization asks one question repeatedly: does this sample point lie inside this triangle? Everything else in the stage — interpolation, fill rules, hierarchical traversal — is built from the machinery that answers it.

1.1 Edge functions

For a directed edge from **a** to **b** and a sample point **p**, define:

$$E_{ab}(\mathbf{p}) = (\mathbf{p}_x - \mathbf{a}_x)(\mathbf{b}_y - \mathbf{a}_y) - (\mathbf{p}_y - \mathbf{a}_y)(\mathbf{b}_x - \mathbf{a}_x)$$

This is the z-component of the cross product of $(\mathbf{b} - \mathbf{a})$ with $(\mathbf{p} - \mathbf{a})$ — the 2D cross product. Its sign tells you which side of the directed line **p** falls on, and its magnitude is twice the area of triangle *abp*. A point is inside the triangle exactly when the three edge functions, taken with consistent winding, all have the same sign.

ONE EXPRESSION, FOUR JOBS

Sign → inside/outside test.

Sign of the sum → winding order, hence backface culling.

Magnitude → twice the sub-triangle area.

Normalized magnitude → barycentric coordinates, hence attribute interpolation.

This is why edge functions, rather than scanline algorithms, are how modern rasterization is done.

1.2 Sample position

Pixels are areas, but we test points. The convention is to sample at the pixel **centre**: pixel (x, y) is tested at $(x + 0.5, y + 0.5)$. The specific convention matters less than its *consistency*: if two triangles sharing an edge are tested at even slightly different points, the shared edge will leak.

2 Fill rules

Consider two triangles sharing an edge, and a sample that lands exactly on it. Both triangles compute $E = 0$ for that edge — not approximately zero, but exactly, because they share vertices and therefore perform identical arithmetic. Floating-point tolerance does not help; the values genuinely are equal.

If the test is ≥ 0 for both, both triangles claim the sample. The fragment is shaded twice: wasted work always, and a visible bright seam wherever blending is enabled. If the test is > 0 for both, neither claims it, and a one-pixel crack of background shows through what should be a solid surface.

2.1 The top-left rule

Make the inclusion decision depend on the *edge*, not on the triangle. An on-edge sample belongs to the triangle if the edge in question is a top edge or a left edge:

- **Top edge**: exactly horizontal, with the triangle's interior below it.
- **Left edge**: an edge that goes downward in screen space, with the interior to its right.

Because the two triangles traverse the shared edge in opposite directions, the edge is top-or-left for exactly one of them. Exactly one claims the sample. No gaps, no double shading, and the result is deterministic and order-independent. This rule is normative in both the Direct3D and Vulkan specifications; GPUs implement it in hardware.

```
bool is_top_left(vec2 a, vec2 b) {
    bool top  = (a.y == b.y) && (b.x < a.x); // horizontal, interior below
    bool left = (b.y > a.y);                 // descending edge
    return top || left;
}

// Classify once per triangle; apply as a bias so the inner loop
// stays branch-free. With integer coordinates EPS is exactly 1.
float b0 = is_top_left(v1, v2) ? 0 : -EPS;
float b1 = is_top_left(v2, v0) ? 0 : -EPS;
float b2 = is_top_left(v0, v1) ? 0 : -EPS;

if (w0 + b0 >= 0 && w1 + b1 >= 0 && w2 + b2 >= 0)
    emit_fragment(...);
```

WHY HARDWARE RASTERIZES IN FIXED POINT

With floating-point coordinates the 'exactly on the edge' case is fragile and the bias is a fudge. With fixed-point coordinates — typically 8 sub-pixel fractional bits — the comparison is exact and the rule is watertight.

The cost is a bound on coordinate range, which is why extremely large triangles can show vertex snapping. This is a real constraint in engines, not a curiosity.

3 Barycentric coordinates

Having computed the three edge functions for the coverage test, normalize them:

$$\lambda_i = E_i(p) / (E_0 + E_1 + E_2)$$

The denominator is twice the full triangle area and is constant per triangle, so this costs one reciprocal per triangle and one multiply per weight. The resulting weights are non-negative inside the triangle and sum to one — which, as Module 02 noted, is exactly the condition under which a weighted sum of *points* is meaningful.

Any per-vertex attribute now interpolates as $\lambda_0 a_0 + \lambda_1 a_1 + \lambda_2 a_2$. Colour, normal, texture coordinate, tangent, depth — all the same operation. Coverage and interpolation turn out to be a single computation, which is the elegant fact at the centre of this module.

4 Perspective-correct interpolation

4.1 Why naive interpolation fails

A texture coordinate varies linearly over the triangle's surface in 3D. Projection divided every coordinate by w , and division is not an affine map, so linear variation in 3D does not survive as linear variation in 2D. Walking across the triangle in equal screen-space steps corresponds to unequal steps across the actual surface — larger steps where the surface is further away.

Interpolating linearly in screen space therefore stretches the texture incorrectly, and because the error depends on the camera, it changes as you move: the texture appears to swim or slide over the geometry. Shallow-angle floors show it most, and splitting the polygon into many small triangles reduces it — which is why some early software renderers subdivided geometry as a workaround.

4.2 The correction

One quantity does survive projection as an affine function of screen position: $1/w$. So interpolate a/w and $1/w$ linearly, both of which is valid, and divide at the end:

$$\mathbf{a}(p) = [\sum \lambda_i (\mathbf{a}_i / w_i)] / [\sum \lambda_i (1/w_i)]$$

```
float iw0 = 1.0f/c0.w, iw1 = 1.0f/c1.w, iw2 = 1.0f/c2.w;

// per fragment
float iw = l0*iw0 + l1*iw1 + l2*iw2;
float p0 = l0*iw0/iw, p1 = l1*iw1/iw, p2 = l2*iw2/iw;

uv      = p0*uv0 + p1*uv1 + p2*uv2;  // corrected
normal  = p0*n0  + p1*n1  + p2*n2;    // corrected
depth   = l0*z0  + l1*z1  + l2*z2;    // NOT corrected
```

Compute the corrected weights once per fragment and reuse them for every attribute. Interpolating depth with the corrected weights is a real and frequent bug.

DEPTH IS THE EXCEPTION

z_ndc is already the output of the perspective divide. It is therefore *already* an affine function of screen position, and interpolating it with plain screen-space barycentrics is correct.

Apply the perspective correction to depth and you get subtly wrong occlusion, which looks like mild z-fighting and sends people hunting in entirely the wrong place.

Rule: anything that was linear *in 3D* needs correction; anything that is already a post-divide screen quantity does not.

5 Making it fast

The edge function is affine in screen position, so its value at the next pixel differs from the current one by a constant:

$$E(x+1, y) = E(x, y) + (b_y - a_y) \quad E(x, y+1) = E(x, y) + (a_x - b_x)$$

Evaluate the three edge functions once at a corner of the bounding box, then add constants as you walk. Three multiplies and three subtractions per pixel collapse into three additions, and the inner loop vectorises trivially across four or eight pixels at once.

The larger win is hierarchical. Test a block of pixels — 8×8 , say — by evaluating the edge functions at its corners. If all four corners lie outside the same edge, every pixel in the block is outside, and the block is skipped without touching a single pixel. If all four are inside all three edges, the block is entirely covered and the inside test can be skipped per pixel. Only partially covered blocks need per-pixel work, and those are a small fraction of a typical triangle's area. Real GPUs do this in a multi-level hierarchy, and it is a large part of why rasterization is so fast.

6 Artifact table

What you see	Cause	Fix
One-pixel cracks along shared edges	No fill rule, or sample positions differ between triangles.	Implement top-left; sample consistently at pixel centres.
Bright seams on alpha-blended geometry	Shared-edge samples shaded twice.	Same fix. The seam is the double-blend.
Textures swim or slide with camera motion	Screen-space linear interpolation of attributes.	Perspective-correct: interpolate a/w and $1/w$, then divide.
Occlusion subtly wrong, mild z-fighting	Depth perspective-corrected when it should not be.	Interpolate z_ndc with plain screen-space barycentrics.
Thin triangles flicker or disappear	Near-degenerate area; the edge-function sum approaches zero and its sign becomes unstable.	Cull triangles below an area threshold before rasterizing.

What you see	Cause	Fix
Vertices snap as geometry moves slowly	Sub-pixel precision exhausted in fixed point.	Reduce triangle size, or increase fractional bits.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
GAMES101 Lectures 05–06 — Rasterization	Coverage, the inside test, and the introduction to sampling that sets up Module 09.
Fabian Giesen — 'A trip through the Graphics Pipeline' and the rasterization series	The best free writing on how real hardware rasterizes: fill rules, fixed point, hierarchical traversal. Essential reading for this module.
Scratchapixel — Rasterization: a Practical Implementation	Full working code including the perspective-correction derivation.
CMU 15-462 — Drawing a Triangle / Sampling	Frames coverage as a sampling problem, which is the framing Module 09 builds on.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement the edge-function rasterizer with a bounding box. Render a single large triangle and confirm the coverage is exact at the edges.
- 2 Implement the top-left fill rule. Render two triangles sharing an edge, each in a different colour with 50% alpha, and verify at 8× magnification that the shared edge is neither doubled nor cracked. Capture before and after.
- 3 Implement barycentric interpolation and render a triangle with the three vertices coloured red, green, and blue. The result should be a smooth gradient with no banding.
- 4 Implement perspective-correct interpolation. Render a large textured floor plane at a shallow angle, with and without correction, and capture both. This is a required deliverable of Project 1.
- 5 Convert the inner loop to incremental edge stepping and measure the speedup on a 5,000-triangle mesh. Then add 8×8 block rejection and measure again.
- 6 Deliberately apply the perspective correction to depth as well. Describe precisely what goes wrong and why it is easy to misdiagnose.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Write the edge function from memory and state four distinct things its value tells you.
- 2 Why does floating-point tolerance not solve the shared-edge problem?
- 3 State the top-left rule and explain why it guarantees exactly one triangle claims each on-edge sample.
- 4 How are barycentric coordinates obtained from the edge functions, and why is this essentially free?
- 5 Explain why screen-space linear interpolation of texture coordinates is wrong, and name the one quantity that is affine in screen space.
- 6 Which interpolated quantity must *not* be perspective-corrected, and why?
- 7 Describe how block-based hierarchical rasterization reduces per-pixel work, and what determines how much it helps.